

NETAJI SUBHAS UNIVERSITY **OF TECHNOLOGY**



Student Information **Management System**

SEMESTER-3 (BATCH 2024-28) BRANCH- CSE

SUBMITTED TO:
Mr. Anand Gupta

SUBMITTED BY:
Riddhi Khandelwal(2024UCS1503)
Anshita Singh(2024UCS1529)
Tanisha Gautam(2024UCS1536)

Problem Statement

Managing student information in many educational institutions is still done through spreadsheets or manual record-keeping. As the number of students grows, these traditional methods become inefficient, error-prone, and difficult to maintain. Tasks such as tracking student details, managing course registrations, updating records, or generating reports become time-consuming and inconsistent.

To address these challenges, this project aims to design and implement a Student Management System using a Database Management System (DBMS). The system will provide a structured, centralized database for storing, updating, and easily retrieving student information, course details, and enrollment data. It will help reduce redundancy, maintain data accuracy, and simplify overall management for administrators and faculty members.

The goal is to build a database-driven solution that enhances efficiency, supports data integrity, and offers meaningful insights through various SQL queries and reports.

Requirement Analysis

Functional Requirements

A. Student Management

The system should allow adding new student records with details such as:

Student ID

Name

Date of Birth

Gender

Address

City/State

Contact information

Users should be able to update student details when required.

Admins should be able to delete student records.

The system should allow searching for students by name, ID, city, or other attributes.

B. Course / Subject Management

The system should maintain details of all courses/subjects including:

Course ID

Course Name

Users should be able to add, update, or remove course information.

C. Student Enrollment

The system must allow students to enroll in courses.

Each enrollment record must link a valid Student ID with a valid Course ID.

Enrollment must include details like:

Enrollment ID

Date of Enrollment

The system should prevent duplicate or invalid enrollments.

D. Reports / Queries

The system should support frequently needed queries such as:

List of students enrolled in a particular course.

Students grouped by city/state.

Total number of students in each course.

Filtered searches using conditions (e.g., students living in a certain state).

Displaying detailed information using JOIN operations.

Aggregate reports using COUNT, AVG, MAX, MIN, etc.

E. Database Setup

Provide SQL scripts to create tables (DDL).

Insert sample data (DML).

Include integrity constraints such as:

Primary Keys

Foreign Keys

Unique and NOT NULL constraints

Non-Functional Requirements

A. Data Integrity

The database must avoid redundancy using proper normalization.

Foreign keys should ensure that data remains consistent and linked correctly.

B. Usability

The schema must be easy to understand and maintain.

Records should be simple to insert, update, and retrieve.

C. Performance

Queries should execute efficiently and return results promptly.

Indexing may be applied on frequently accessed fields.

D. Scalability

The database structure should allow adding future modules such as:

Attendance

Grades

Faculty Management

without major structural changes.

E. Portability

The SQL scripts should run on common relational database systems such as MySQL, PostgreSQL, or SQL Server with minimal changes.

Scope of the Project

The system focuses mainly on students, courses, and enrollment management.

It does not include deep ERP-level features (such as payroll or library management).

It is designed for academic purposes and can be extended later.

Constraints

The database assumes valid user input (e.g., correct formats for email or phone numbers).

Entities & Attributes

Student

student_id — Primary key (or unique identifier) for each student

first_name — Student's first name

last_name — Student's last name

gender — Gender of the student

dob — Date of birth

contact_no — Phone / contact number

email — Email address

address — Street / house address

city — City name

state — State name

zip_code — Zip or postal code

enrollment_date — Date when the student joined / was registered

Course (or Subject)

course_id — Unique identifier for each course

course_name — Name/title of the course

duration — Duration of the course (could be in months / years / hours)

fee — Course fee (if maintained)

Enrollment

enrollment_id — Unique key for enrollment record

student_id — Foreign key referencing Student

course_id — Foreign key referencing Course

enroll_date — Date of enrollment

Relationships between Entitie

Relationships between Entities

Student ↔ Enrollment: One-to-many — A student can enroll in multiple courses.

Course ↔ Enrollment: One-to-many — A course can have many students enrolling.

Implied many-to-many relationship between Student and Course via the Enrollment table.

Functional Dependency (FD)

A Functional Dependency (FD) is a relationship between two sets of attributes in a database. It describes how one attribute uniquely determines another attribute. In simple terms: If the value of attribute A determines the value of attribute B, then B is functionally dependent on A.

$A \rightarrow B$ (Read as: “A functionally determines B”). For every value of A, there is only one corresponding value of B. Two rows that have the same value of A must have the same value of B.

Importance of Functional Dependencies: Functional dependencies help to remove redundancy, prevent update, insert, and delete anomalies, normalize database tables (1NF, 2NF, 3NF, BCNF), identify primary keys and candidate keys, and maintain data accuracy and integrity.

Functional Dependencies for the Database

1. STUDENT Table

Attributes: (id, rollno, sname, sem, gender, branch, email, number, address)

Functional Dependencies:

1. $\text{id} \rightarrow \text{rollno}, \text{sname}, \text{sem}, \text{gender}, \text{branch}, \text{email}, \text{number}, \text{address}$
2. $\text{rollno} \rightarrow \text{id}, \text{sname}, \text{sem}, \text{gender}, \text{branch}, \text{email}, \text{number}, \text{address}$

2. ATTENDANCE Table

Attributes: (aid, rollno, attendance)

Functional Dependencies:

1. $\text{aid} \rightarrow \text{rollno}, \text{attendance}$

3. DEPARTMENT Table

Attributes: (cid, branch)

Functional Dependencies:

1. $\text{cid} \rightarrow \text{branch}$
2. $\text{branch} \rightarrow \text{cid}$

Creating Tables using Python:-

Code Blame 255 lines (205 loc) · 8.21 KB

```
1  from flask import Flask,render_template,request,session,redirect,url_for,flash
2  from flask_sqlalchemy import SQLAlchemy
3  from flask_login import UserMixin
4  from werkzeug.security import generate_password_hash,check_password_hash
5  from flask_login import login_user,logout_user,login_manager,LoginManager
6  from flask_login import login_required,current_user
7  import json
8
9  # MY db connection
10 local_server= True
11 app = Flask(__name__)
12 app.secret_key='tanishagautam'
13
14
15 # this is for getting unique user access
16 login_manager=LoginManager(app)
17 login_manager.login_view='login'
18
19 @login_manager.user_loader
20 def load_user(user_id):
21     return User.query.get(int(user_id))
22
23
24
25 # app.config['SQLALCHEMY_DATABASE_URL']='mysql://username:password@localhost/databas_ta
26 app.config['SQLALCHEMY_DATABASE_URI']='mysql://root:@localhost/DBMSProject'
27 db=SQLAlchemy(app)
28
29 # here we will create db models that is tables
30 class Test(db.Model):
31     id=db.Column(db.Integer,primary_key=True)
32     name=db.Column(db.String(100))
33     email=db.Column(db.String(100))
34
35 class Department(db.Model):
36     cid=db.Column(db.Integer,primary_key=True)
37     branch=db.Column(db.String(100))
38
```

```
class Attendance(db.Model):
    aid=db.Column(db.Integer,primary_key=True)
    rollno=db.Column(db.String(100))
    attendance=db.Column(db.Integer())
```

✓

```
class Trig(db.Model):
    tid=db.Column(db.Integer,primary_key=True)
    rollno=db.Column(db.String(100))
    action=db.Column(db.String(100))
    timestamp=db.Column(db.String(100))
```

✓

```
class User(UserMixin,db.Model):
    id=db.Column(db.Integer,primary_key=True)
    username=db.Column(db.String(50))
    email=db.Column(db.String(50),unique=True)
    password=db.Column(db.String(1000))
```

✓

```
class Student(db.Model):
    id=db.Column(db.Integer,primary_key=True)
    rollno=db.Column(db.String(50))
    sname=db.Column(db.String(50))
    sem=db.Column(db.Integer)
    gender=db.Column(db.String(50))
    branch=db.Column(db.String(50))
    email=db.Column(db.String(50))
    number=db.Column(db.String(12))
    address=db.Column(db.String(100))
```

```

61     class Student(db.Model):
75         return render_template('index.html')
76
77     @app.route('/studentdetails')
78     def studentdetails():
79         # query=db.engine.execute(f"SELECT * FROM `student`")
80         query=Student.query.all()
81         return render_template('studentdetails.html',query=query)
82
83     @app.route('/triggers')
84     def triggers():
85         # query=db.engine.execute(f"SELECT * FROM `trig`")
86         query=Trig.query.all()
87         return render_template('triggers.html',query=query)
88
89     @app.route('/department',methods=['POST','GET'])
90     def department():
91         if request.method=="POST":
92             dept=request.form.get('dept')
93             query=Department.query.filter_by(branch=dept).first()
94             if query:
95                 flash("Department Already Exist","warning")
96                 return redirect('/department')
97             dep=Department(branch=dept)
98             db.session.add(dep)
99             db.session.commit()
00             flash("Department Addes","success")
01             return render_template('department.html')
02
03     @app.route('/addattendance',methods=['POST','GET'])
04     def addattendance():
05         # query=db.engine.execute(f"SELECT * FROM `student`")
06         query=Student.query.all()
07         if request.method=="POST":
08             rollno=request.form.get('rollno')
09             attend=request.form.get('attend')
10             print(attend.rollno)

```

```

61     class Student(db.Model):
75         return render_template('index.html')
76
77     @app.route('/studentdetails')
78     def studentdetails():
79         # query=db.engine.execute(f"SELECT * FROM `student`")
80         query=Student.query.all()
81         return render_template('studentdetails.html',query=query)
82
83     @app.route('/triggers')
84     def triggers():
85         # query=db.engine.execute(f"SELECT * FROM `trig`")
86         query=Trig.query.all()
87         return render_template('triggers.html',query=query)
88
89     @app.route('/department',methods=['POST','GET'])
90     def department():
91         if request.method=="POST":
92             dept=request.form.get('dept')
93             query=Department.query.filter_by(branch=dept).first()
94             if query:
95                 flash("Department Already Exist","warning")
96                 return redirect('/department')
97             dep=Department(branch=dept)
98             db.session.add(dep)
99             db.session.commit()
00             flash("Department Addes","success")
01             return render_template('department.html')
02
03     @app.route('/addattendance',methods=['POST','GET'])
04     def addattendance():
05         # query=db.engine.execute(f"SELECT * FROM `student`")
06         query=Student.query.all()
07         if request.method=="POST":
08             rollno=request.form.get('rollno')
09             attend=request.form.get('attend')
10             print(attend.rollno)

```

```

61     class Student(db.Model):
75         return render_template('index.html')
76
77     @app.route('/studentdetails')
78     def studentdetails():
79         # query=db.engine.execute(f"SELECT * FROM `student`")
80         query=Student.query.all()
81         return render_template('studentdetails.html',query=query)
82
83     @app.route('/triggers')
84     def triggers():
85         # query=db.engine.execute(f"SELECT * FROM `trig`")
86         query=Trig.query.all()
87         return render_template('triggers.html',query=query)
88
89     @app.route('/department',methods=['POST','GET'])
90     def department():
91         if request.method=="POST":
92             dept=request.form.get('dept')
93             query=Department.query.filter_by(branch=dept).first()
94             if query:
95                 flash("Department Already Exist","warning")
96                 return redirect('/department')
97             dep=Department(branch=dept)
98             db.session.add(dep)
99             db.session.commit()
00             flash("Department Addes","success")
01             return render_template('department.html')
02
03     @app.route('/addattendance',methods=['POST','GET'])
04     def addattendance():
05         # query=db.engine.execute(f"SELECT * FROM `student`")
06         query=Student.query.all()
07         if request.method=="POST":
08             rollno=request.form.get('rollno')
09             attend=request.form.get('attend')
10             print(attend.rollno)

```

```
189         flash("Signup Succes Please Login","success")
190         return render_template('login.html')
191
192
193
194         return render_template('signup.html')
195
196     @app.route('/login',methods=['POST','GET'])
197     ✓ def login():
198         if request.method == "POST":
199             email=request.form.get('email')
200             password=request.form.get('password')
201             user=User.query.filter_by(email=email).first()
202
203             # if user and check_password_hash(user.password,password):
204             if user and user.password == password:
205                 login_user(user)
206                 flash("Login Success","primary")
207                 return redirect(url_for('index'))
208             else:
209                 flash("invalid credentials","danger")
210                 return render_template('login.html')
211
212         return render_template('login.html')
213
214     @app.route('/logout')
215     @login_required
216     def logout():
217         logout_user()
218         flash("Logout SuccessFul","warning")
219         return redirect(url_for('login'))
220
221
222
223     @app.route('/addstudent',methods=['POST','GET'])
```

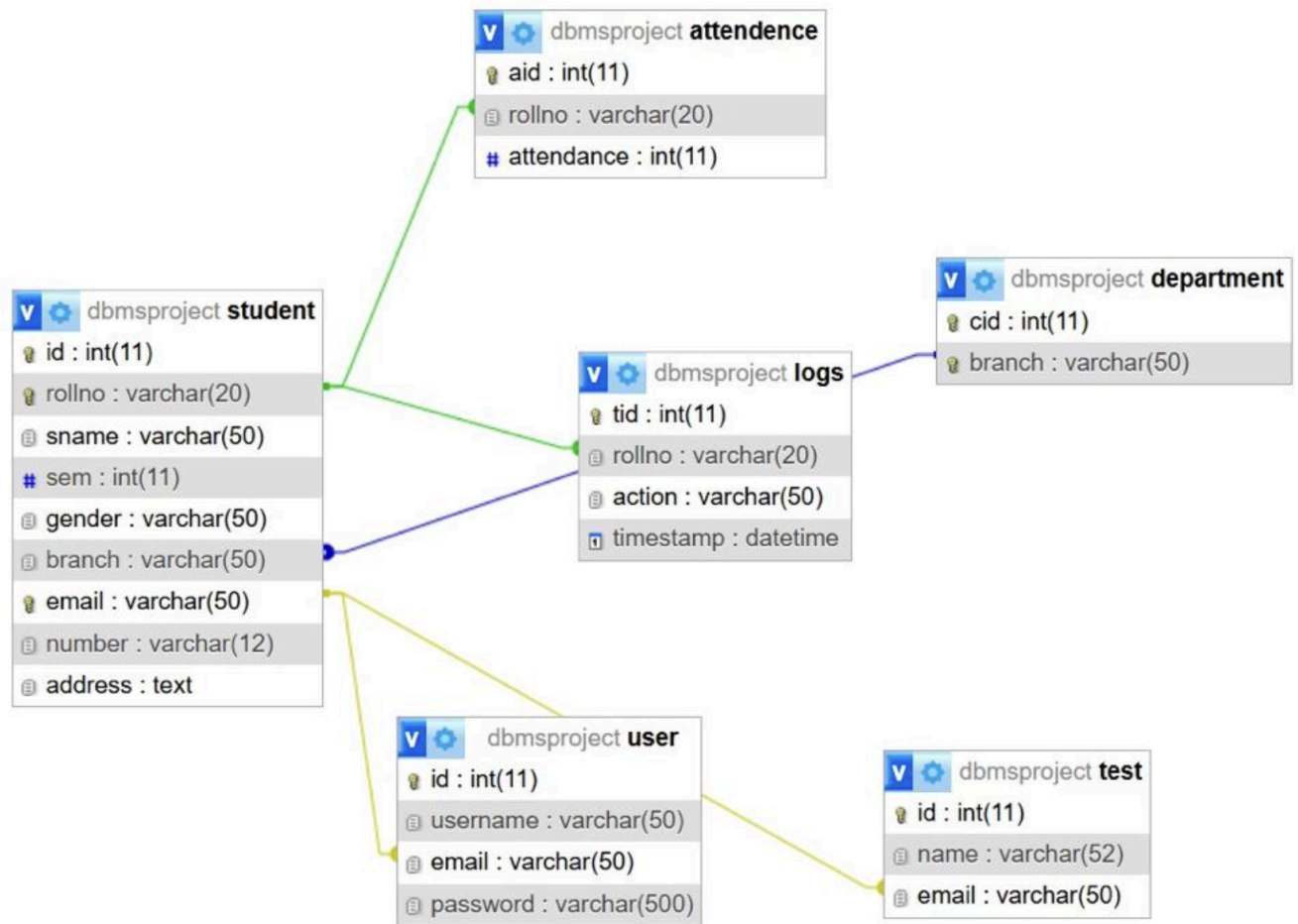
```

224 @login_required
225 def addstudent():
226     # dept=db.engine.execute("SELECT * FROM `department`")
227     dept=Department.query.all()
228     if request.method=="POST":
229         rollno=request.form.get('rollno')
230         sname=request.form.get('sname')
231         sem=request.form.get('sem')
232         gender=request.form.get('gender')
233         branch=request.form.get('branch')
234         email=request.form.get('email')
235         num=request.form.get('num')
236         address=request.form.get('address')
237         # query=db.engine.execute(f"INSERT INTO `student` (`rollno`,`sname`,`sem`,`gender`,`branch`,`email`,`number`,`address`) VALUES ({rollno},{sname},{sem},{gender},{branch},{email},{num},{address})")
238         query=Student(rollno=rollno,sname=sname,sem=sem,gender=gender,branch=branch,email=email,number=num,address=address)
239         db.session.add(query)
240         db.session.commit()
241
242         flash("Booking Confirmed","info")
243
244
245     return render_template('student.html',dept=dept)
246 @app.route('/test')
247 def test():
248     try:
249         Test.query.all()
250         return 'My database is Connected'
251     except:
252         return 'My db is not Connected'
253
254
255 app.run(debug=True)

```

Table Layout :-

Table	Action	Rows	Type	Collation	Size	Overhead
☐ attendance	★ Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ department	★ Browse Structure Search Insert Empty Drop	7	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ student	★ Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ test	★ Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ trig	★ Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ user	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	16.0 KiB	-
6 tables	Sum	18	InnoDB	utf8mb4_general_ci	96.0 KiB	0 B



SQL queries questions

```
SET SQL_MODE = "NO_AUTO_VALUE_ON_ZERO";
START TRANSACTION;
SET time_zone = "+00:00";

/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
/*!40101 SET NAMES utf8mb4 */;

--
-- Database: `students`
--

--
-- Table structure for table `attendance`
--

CREATE TABLE `attendance` (
  `aid` int(11) NOT NULL,
  `rollno` varchar(20) NOT NULL,
  `attendance` int(100) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

--
-- Dumping data for table `attendance`
--

INSERT INTO `attendance` (`aid`, `rollno`, `attendance`) VALUES
(6, '1ve17cs012', 98);
```

```

CREATE TRIGGER `DELETE` BEFORE DELETE ON `student` FOR EACH ROW INSERT INTO trig VALUES(null,OLD.rollno,'STUDENT DELETED'
$$
DELIMITER ;
DELIMITER $$
CREATE TRIGGER `Insert` AFTER INSERT ON `student` FOR EACH ROW INSERT INTO trig VALUES(null,NEW.rollno,'STUDENT INSERTED'
$$
DELIMITER ;
DELIMITER $$
CREATE TRIGGER `UPDATE` AFTER UPDATE ON `student` FOR EACH ROW INSERT INTO trig VALUES(null,NEW.rollno,'STUDENT UPDATED',
$$
DELIMITER ;

-- -----

--
-- Table structure for table `test`
--

CREATE TABLE `test` (
  `id` int(11) NOT NULL,
  `name` varchar(52) NOT NULL,
  `email` varchar(50) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

--
-- Dumping data for table `test`
--

INSERT INTO `test` (`id`, `name`, `email`) VALUES
(1, 'aaa', 'aaa@gmail.com');

```

```
CREATE TABLE `user` (  
  `id` int(11) NOT NULL,  
  `username` varchar(50) NOT NULL,  
  `email` varchar(50) NOT NULL,  
  `password` varchar(500) NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;  
  
--  
-- Dumping data for table `user`  
--  
  
INSERT INTO `user` (`id`, `username`, `email`, `password`) VALUES  
(4, 'Suhani', 'Suhani123@gmail.com', 'pbkdf2:sha256:150000$1CSLss89$  
  
--  
-- Indexes for dumped tables  
--  
  
--  
-- Indexes for table `attendance`  
--  
ALTER TABLE `attendance`  
  ADD PRIMARY KEY (`aid`);  
  
--  
-- Indexes for table `department`  
--  
ALTER TABLE `department`  
  ADD PRIMARY KEY (`cid`);  
  
--  
-- Indexes for table `student`  
--  
ALTER TABLE `student`  
  ADD PRIMARY KEY (`id`);
```

```
CREATE TABLE `department` (  
  `cid` int(11) NOT NULL,  
  `branch` varchar(50) NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
-- Dumping data for table `department`  
--
```

```
INSERT INTO `department` (`cid`, `branch`) VALUES  
(2, 'Information Science'),  
(3, 'Electronic and Communication'),  
(4, 'Electrical & Electronic'),  
(5, 'Civil '),  
(7, 'computer science'),  
(8, 'IOT');
```

```
-- Table structure for table `student`  
--
```

```
CREATE TABLE `student` (  
  `id` int(11) NOT NULL,  
  `rollno` varchar(20) NOT NULL,  
  `sname` varchar(50) NOT NULL,  
  `sem` int(20) NOT NULL,  
  `gender` varchar(50) NOT NULL,  
  `branch` varchar(50) NOT NULL,  
  `email` varchar(50) NOT NULL,  
  `number` varchar(12) NOT NULL,  
  `address` text NOT NULL
```

```
ALTER TABLE `attendance`
  MODIFY `aid` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=7;

--
-- AUTO_INCREMENT for table `department`
--
ALTER TABLE `department`
  MODIFY `cid` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=9;

--
-- AUTO_INCREMENT for table `student`
--
ALTER TABLE `student`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=7;

--
-- AUTO_INCREMENT for table `test`
--
ALTER TABLE `test`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=2;

--
-- AUTO_INCREMENT for table `trig`
--
ALTER TABLE `trig`
  MODIFY `tid` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=10;

--
-- AUTO_INCREMENT for table `user`
--
ALTER TABLE `user`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=5;
COMMIT;

/*!40101 SET CHARACTER_SET_CLIENT=@OLD_CHARACTER_SET_CLIENT */;
```

E-R Model:-

